

PS-LAB6-289810

Zadanie 1: Zmienne

Napisz skrypt wymagający podania długości i wysokości trójkąta w argumencie, obliczający pole powierzchni trójkąta. Wynik wyświetl na ekranie.

```
[Int]$dlugosc = $args[0] #wczytujemy argument do zmiennej i upewniamy sie ze jest on INT
[Int]$wysokosc = $args[1]
$Pole = ($dlugosc*$wysokosc)/2 #Obliczmy pole i zapisujemy je w zmiennej
Write-Host "Pole trójkąta: $Pole" #wypisujemy wynik
```

Zadanie 2: Instrukcje warunkowe

Napisz skrypt, który zapyta użytkownika o liczbę i sprawdzi, czy jest większa od 10. Jeśli tak, wyświetli komunikat "Liczba jest większa od 10", w przeciwnym razie wyświetli komunikat "Liczba jest mniejsza od 10".

```
[Int]$Liczba = Read-Host -Prompt "Podaj liczbę: " #wczytujemy tekst z konsoli i upewniamy sie ze jest to liczba, a następnie zapisujemy ja w zmiennej

#sprawdzamy czy liczba jest wieksza od 10
if ($Liczba -gt 10) {
    Write-Host "Liczba jest wieksza od 10" #jesli jest to wypisujemy do konsoli dany tekst
} else {
    Write-Host "Liczba jest mniejsza od 10" # jesli nie jest to wypisujemy dany tekst
}
```

Zadanie 3: Operatory logiczne

Napisz skrypt, który poprosi użytkownika o podanie nazwy użytkownika i hasła, a następnie sprawdzi, czy podane dane są poprawne (np. nazwa użytkownika to "admin" i hasło to "password"). **Zastosuj maskowanie wprowadzanego hasła.**

```
$Uzytkownik = Read-Host -Prompt "Podaj nazwę użytkownika: " #wczytujemy tekst z konsoli

$ZaszyfrowaneHaslo = Read-Host -AsSecureString -Prompt "Podaj hasło: " #wczytujemy
```

tekst z konsoli, a następnie przechowujemy go jako SecureText

#zeby odzyskac nasze zaszyfrowane haslo trzeba utworzyc nowy obiekt PSCredential i podac co kolwiek do pola Login i nasze zaszyfrowane haslo do pola z haslem
#nastepnie uzywamy .GetNetworkCredential().Password by dostac nasze haslo jako czysty tekst

```
$Haslo = (New-Object System.Management.Automation.PSCredential($Uzytkownik,  
$ZaszyfrowaneHaslo)).GetNetworkCredential().Password
```

#sprawdzamy czy haslo jest zgodne jak i rowniez nazwa uzytkownika

```
if ($Uzytkownik -eq "admin" -and $Haslo -eq "password") {  
    Write-Host "Poprawne logowanie" # wypisujemy to jesli jest poprawnie  
} else {  
    Write-Host "Bledne haslo lub login" # wypisujemy to jesli nie jest poprawnie  
}
```

Zadanie 4: Pętle

Napisz skrypt generujący 10 adresów IP za pomocą pętli, które będą

różniły się czwartym oktetem rosnąco w adresie 192.168.1.X

```
$Baza = "192.168.0" #baza naszego ip
```

#nasza minimalna wartość dla ostatniego oktetu

```
[Int]$Min = 0
```

#pętla która powtórzy się 10razy, i wygeneruje nam 10 ip

```
for ([Int]$i=1;$i -le 11; $i++){
```

#wybermy losową liczbę z zakresu \$min do \$min+\$i | dla pierwszego powtórzenia to będzie \$min=0 \$min+\$i=1

```
    $Random = Get-Random -Minimum $Min -Maximum ($Min + $i)
```

#wypisujemy nasze nowe ip

```
    Write-Host "$Baza.$Random"
```

#dodajmy nasze \$i do \$min

```
    $Min += $i
```

```
}
```

Zadanie 5 : Funkcje

Utwórz skrypt wyświetlający informacje o systemie: Aktualna data, Wersja

systemu, Użytkownik, AdresIP.

Każda informacja powinna znajdować się w osobnej funkcji. Użyj jednej zmiennej z nazwą komputera w każdej funkcji.

Przykład: „Aktualna data na (Nazwa komputera) to X”

```
#funkcja która zwraca aktualną datę z komputera
function Aktualna_Data {
    return Get-Date
}

#funkcja która zwraca aktualną wersję systemu
function Wersja_Systemu {
    return [Environment]::OSVersion
}

#funkcja która zwraca nazwę aktualnego użytkownika
function Uzytkownik {
    return $env:USERNAME
}

#funkcja która zwraca IP komputera
function Adres_IP {
    #pobiermy table z adresami dla ipv4 następnie wybierzmy tylko InterfaceAlias
    #i IP Address, a następnie zamienmy wszystko na string Out-String'em
    return Get-NetIPAddress -AddressFamily IPv4 | Select-Object InterfaceAlias,
    IP Address | Out-String
}

#pobierzmy nazwę komputera z zmiennych środowiskowych
$ComputerName = $env:COMPUTERNAME

#wypiszmy wszystkie funkcje
Write-Host "Aktualna data na komputerze $ComputerName to: $(Aktualna_Data)"
Write-Host "Aktualna wersja systemu na komputerze $ComputerName to: `n
$(Wersja_Systemu)"
Write-Host "Użytkownik który jest aktualnie aktywny na komputerze $ComputerName to:
$(Uzytkownik)"
Write-Host "Dane ipv4 dla komputera $ComputerName : `n $(Adres_IP)"
```

6. Zadania połączone

Utwórz plik CSV o nazwie `input_file.csv` z zawartością:

```
Component;GenerateReport
Computername;True
Manufacturer;True
Model;True
SerialNumber;True
CpuName;False
RAM;True
```

Napisz skrypt PowerShell, który będzie:

- odczytywać zawartość pliku input_file.csv
- wykonywać iterację po wszystkich wierszach w pliku input_file.csv i zwróci informacje o komputerze/laptopie tylko wtedy, gdy w danym wierszu opcja GenerateReport ma wartość True.

Dane wyjściowe powinny wyglądać następująco:

```
Computername: ABWI45774RZ
Manufacturer: LENOVO
Model: ThinkPad T123
SerialNumber: M4564XDCG54DSA
RAM: 32 GB
```

Zauważ, że CpuName nie jest wyświetlane, ponieważ GenerateReport jest ustawione na False w tym wierszu. Spróbuj zmienić wartości True|False w kolumnie GenerateReport pliku input_file.csv, wykonaj skrypt ponownie.

Komenda Get-ComputerInfo jest Twoim przyjacielem

```
#wczytujemy plik csv i ustawiamy delimiter jako ;
$file = Import-Csv -Delimiter ";" .\input_file.csv

#zapisujemy Get-ComputerInfo do zmiennej by usprawnić wczytywanie infoamcji o
komputerze
$info = Get-ComputerInfo

Write-Host "RAPORT"

#iterujemy nasz plik csv po każdym wierszu
foreach ($item in $file){
    #sprawdza czy przy danym wierszu ma coś wartość true a następnie genruje raport
    na tej podstawie
    #$.Component jest to np.Computername | $.GenerateReport jest to true
```

```

Lub false
if ($item.Component -eq "Computername" -and $item.GenerateReport -eq $true){
    #$env:COMPUTERNAME - nazwa komputera w którym skrypt jest puszczony
    Write-Host "Computer Name: $env:COMPUTERNAME"
}
elseif ($item.Component -eq "Manufacturer" -and $item.GenerateReport -eq $true)
{
    #$info.CsManufacturer - bierze z get-computerinfo tylko wartosc z zmiennej
CsManufacturer
    Write-Host "Manufacturer: $($info.CsManufacturer)"
}
elseif ($item.Component -eq "Model" -and $item.GenerateReport -eq $true){
    #$info.CsSystemFamily - bierze z get-computerinfo tylko wartosc z zmiennej
CsSystemFamily
    Write-Host "Model: $($info.CsSystemFamily)"
}
elseif ($item.Component -eq "SerialNumber" -and $item.GenerateReport -eq $true)
{
    #$info.OsSerialNumber - bierze z get-computerinfo tylko wartosc z zmiennej
OsSerialNumber
    Write-Host "OsSerialNumber: $($info.OsSerialNumber)"
}
elseif ($item.Component -eq "CpuName" -and $item.GenerateReport -eq $true){
    #$info.CsProcessors - bierze z get-computerinfo tylko wartosc z zmiennej
CsProcessors
    Write-Host "CpuName: $($info.CsProcessors)"
}
elseif ($item.Component -eq "RAM" -and $item.GenerateReport -eq $true){
    #$info.CsTotalPhysicalMemory - bierze z get-computerinfo tylko wartosc z
zmiennej CsTotalPhysicalMemory i dzielimy go przez 1GB poniewaz jest zapisany w
bitach
    Write-Host "RAM: $($info.CsTotalPhysicalMemory / 1GB)GB"
}
}

```

Zadanie dodatkowe

Zapisz dane wyjściowe z zadanie 6 do pliku. Nazwa pliku powinna być jak w przykładzie: "ComputerReport_YYYYMMdd-HH:mm:ss.txt", gdzie:

- RRRR - bieżący rok

- MM - bieżący miesiąc
- dd - bieżący dzień
- HH - bieżąca godzina
- mm - bieżąca minuta
- ss - bieżąca sekunda

Skrypt powinien wyświetlić komunikat w konsoli:

Raport został zapisany w pliku: <pełna ścieżka do pliku>.

```
#niebiede tutaj komentować starych rzeczy bo skrypt jest prawnie taki sam jak z skrypt6.ps1
```

```
$file = Import-Csv -Delimiter ";" .\input_file.csv
$info = Get-ComputerInfo
```

```
#Nazwa pliku w którym zostanie zapisany raport
#zeby sformatować Get-Date używamy ToString z takim kodowaniem y-lata M-miesiąc d-dzień H-godziny m-minuty s-sekundy
```

```
$raport_name = "ComputerRaport_$((Get-Date).ToString("yyyyMMdd-HHmms")).txt"
```

```
#w taki sposób zapisujemy co kolwiek do pliku, tak samo jak do konsoli tylko jeszcze trzeba pipe'nąć to do Out-File
```

```
#zapisujemy "RAPORT" do pliku bez nadpisania pliku bo chcemy aby RAPORT był zapisywany od początku pliku
```

```
Write-Output "RAPORT" | Out-File $raport_name
```

```
foreach ($item in $file){
    if ($item.Component -eq "Computername" -and $item.GenerateReport -eq $true){
        #zapisujemy $env:COMPUTERNAME do pliku z nadpisaniem zeby nie usunąć zapisengo tekstu w pliku
        Write-Output "Computer Name: $env:COMPUTERNAME" | Out-File $raport_name -Append
    }
    elseif ($item.Component -eq "Manufacturer" -and $item.GenerateReport -eq $true)
    {
        #zapisujemy $info.CsManufacturer do pliku z nadpisaniem zeby nie usunąć zapisengo tekstu w pliku
        Write-Output "Manufacturer: $($info.CsManufacturer)" | Out-File $raport_name -Append
    }
    elseif ($item.Component -eq "Model" -and $item.GenerateReport -eq $true){
        #zapisujemy $info.CsSystemFamily do pliku z nadpisaniem zeby nie usunąć
```

zapisengo tekstu w pliku

```
        Write-Output "Model: $($info.CsSystemFamily)" | Out-File $raport_name -
Append
    }
    elseif ($item.Component -eq "SerialNumber" -and $item.GenerateReport -eq $true)
    {
        #zapsiujemy $info.OsSerialNumber do pliku z nadpisaniem zeby nie usunąć
        zapisengo tekstu w pliku
        Write-Output "OsSerialNumber: $($info.OsSerialNumber)" | Out-File
$raport_name -Append
    }
    elseif ($item.Component -eq "CpuName" -and $item.GenerateReport -eq $true){
        #zapsiujemy $info.CsProcessors do pliku z nadpisaniem zeby nie usunąć
        zapisengo tekstu w pliku
        Write-Output "CpuName: $($info.CsProcessors)" | Out-File $raport_name -
Append
    }
    elseif ($item.Component -eq "RAM" -and $item.GenerateReport -eq $true){
        #zapsiujemy $info.CsTotalPhysicalMemory do pliku z nadpisaniem zeby nie
        usunąć zapisengo tekstu w pliku
        Write-Output "RAM: $($info.CsTotalPhysicalMemory / 1GB)GB" | Out-File
$raport_name -Append
    }
}
```

#Get-Location - daje nam lokalizacje gdzie skrypt został uruchomiony

```
Write-Host "Raport został zapisany w pliku: $(Get-Location)\$raport_name"
```